

Data Dependence and Data-Flow Analysis of Arrays

Dror E. Maydan, Saman P. Amarasinghe and Monica S. Lam
Computer Systems Laboratory
Stanford University, CA 94305

1 Introduction

The power of any compiler is derived from, and also limited by, its program analyzers. Finding the right abstraction for program analysis is crucial in the development of compiler technology. For the abstraction to be useful, it must include sufficient information to support the code optimizations and transformations. In addition, it must be tractable to extract the information from at least a large enough set of programs. Data dependence, distance vectors and direction vectors are important data abstractions that have been proven useful for parallelization and loop transformations, and they are applicable to a large set of programs.

The parallelizing compiler research frontier is currently pushing the limits of traditional data dependence analysis. For example, evaluation of today's compiler technology suggests that reuse of data arrays in programs greatly reduces the opportunities for parallelism. Many more loops can be parallelized by *privatizing* the work arrays, that is, assigning a separate copy of the array to each processor. The information needed to support such an optimization is not just *memory disambiguation*, that is, if two references can refer to the same location. To determine if each iteration can have its own copy of the array, we need *data-flow analysis* on individual array elements. Another important research topic is code generation for distributed memory machines. The compiler is responsible for maintaining the coherence of data across processors. Generating efficient distributed memory code requires exact data-flow relationships between accesses to individual array locations. This paper proposes several analysis techniques that are useful for higher level optimizations.

As we develop new data dependence abstractions, it is useful to relate the different abstractions in a uniform mathematical framework. We show that the various existing data dependence abstractions can be viewed as simply different approximations to the same dependence problem. For example, we say that two references are dependent as long as one dynamic pair of the instances of the two references can refer to the same location. Dependence levels, direction vectors and distance vectors provide more accurate approximations. We can define the approximations precisely using the mathematical concept of equivalence classes. *Simple data dependence* is a lower limit of the approximation in that all pairs of instances are said to belong to the same single equivalence class. The upper

This research was supported in part by DARPA contract N00039-91-C-0138 and by fellowships from AT&T Bell Laboratories and Intel Corporation.

limit of the approximation is to find out precisely the dependence between every pair of instances. In this limit, every pair belongs to its own equivalence class. The first analysis technique introduced in this paper, *precise data dependence pairs*, addresses that problem. The techniques of *dependence levels*, *distance vectors* and *direction vectors* all correspond to different intermediate partitions of the problem space into different equivalence classes.

Our second set of dependence abstractions introduce the consideration of data flow into the previous data dependence problems. More specifically, a read operation does not have a *flow dependence* on a write operation to the same location, if there is an intervening write to the same location. The flow dependence abstractions in this paper correspond to different approximations of the problem of flow dependence. We call them *simple data-flow dependence*, *data-flow levels*, *data-flow distance vectors*, *data-flow direction vectors* and *precise data-flow dependence pairs* analogous to their counterparts without data-flow consideration.

In this paper, we start by presenting the unified mathematical framework for the various dependence abstractions, and introduce the new analysis techniques within that context. We then describe the algorithms to find the new dependence abstractions proposed. We finally close with an application, array privatization, requiring the deeper analysis described in the paper.

2 A Framework for Array Data Analysis

The traditional problem of data dependence can better be described as memory disambiguation. That is, we say that two array references are data dependent if some instances of the two references can refer to the same location. This is, however, a very crude approximation to the relation between the dynamic instances of the two references. Consider the following simple example:

```
for i
  A[i] = A[i]+B[i];
```

Although both the read and write references to A can refer to the same location, every pair of related instances belongs to the same iteration. This additional information makes it possible for the loop to be parallelized. This information is captured by either distance or direction vectors. That is, distance and direction vectors gives us more resolution over the information on the pairs of related dynamic instances.

We can unify the various dependence analyses using the concept of equivalence classes. Each data dependence abstraction partitions the *pairs of dynamic instances* of two (not necessarily distinct) accesses into equivalence classes. If an equivalence class contains a pair of instances that refer to the same location, the data dependence approximation considers all pairs within the equivalence class to be “data dependent”. The design and number of different equivalence classes defines the precision of the analysis.

As an example, we can partition the space into two sets; one set containing all the pairs such that the iteration number of the two dynamic instances are equal, the other set containing all the pairs such that the iteration numbers are different. If there are any dependent pairs in the first equivalence set, there is a “loop-independent” dependence

between the two references. If there are any dependent pairs in the second set, there is a “loop carried” dependence. Of course, two references may have dependent pairs in one, both or neither of the two equivalence sets.

2.1 Data Dependence Analysis

Definition 2.1 Let $\vec{s}$ be a symbolic constant vector,

B be affine bound constraints,

$I = \left\{ \vec{i} = [i_1, \dots, i_n] \mid \vec{i} \in Z^n \text{ such that } B(\vec{i}, \vec{s}) \geq 0 \right\}$ be the index set of a loop nest,

$\vec{i}, \vec{i}', \vec{j}, \vec{j}' \in I$, be iterations,

F and F' be two affine access functions.

A data dependence abstraction, D , is defined by a equivalence function p on $I \times I$. (That is, $(\vec{i}, \vec{i}')$ and $(\vec{j}, \vec{j}')$ belong to the same equivalence class iff $p(\vec{i}, \vec{i}') = p(\vec{j}, \vec{j}')$.)

Iteration $\vec{i}'$ is data dependent on $\vec{i}$ according to D iff

$$\exists \vec{j}, \vec{j}' \text{ such that } F(\vec{j}, \vec{s}) = F'(\vec{j}', \vec{s}), \vec{j} \text{ executes before } \vec{j}' (\vec{j} \prec \vec{j}'), p(\vec{i}, \vec{i}') = p(\vec{j}, \vec{j}')$$

In the following, we first show how to describe existing data dependence abstractions within this framework, and we introduce a new data abstraction that corresponds to the abstraction with the finest resolution.

- **Simple Data Dependence Analysis.** Under the formulation of simple data dependence analysis, all pairs of dynamic instances simply belong to the same equivalence class. That is, if there is a data dependence between any pair of instances, all pairs of iterations are considered dependent.
- **Dependence Level** partition all the iteration pairs by the outermost loop for which the two iteration are different. That is,

$$P_{level}(i_{1\dots n}, i'_{1\dots n}) = \begin{cases} k & i_{1\dots k-1} = i'_{1\dots k-1} \text{ and } i_k \neq i'_k \\ n+1 & i = i' \end{cases}$$

- **Distance Vectors** partition all the iteration pairs by their differences. That is,

$$P_{dist}(\vec{i}, \vec{i}') = \vec{i}' - \vec{i}$$

- **Direction Vectors** partition all the iteration pairs by the signs of their differences. That is,

$$P_{dir}(\vec{i}, \vec{i}') = \text{sign}(\vec{i}' - \vec{i})$$

- **Precise Data Dependence Pairs** In the extreme case, each pair in the iteration space belongs to its own equivalence class. We can therefore be more precise, a pair of dynamic instances are considered data dependent if they refer to the same location.

2.2 Data Flow Analysis

Data dependence analysis only indicates that two references can refer to the same location; it does not indicate if a particular read actually uses the value of a particular write operation. Consider the following simple example:

```
for i
  A[3] = A[3]+1;
```

Precise data dependence analysis would determine that a read operation in iteration i is data dependent on all write operations in the preceding iterations. However, it is clear that all but the first iteration is reading the data written in the previous iteration. We say that the write in the immediately preceding iteration *covers* the dependence from the other earlier write operations.

This concept of coverage is orthogonal to the concept of approximation. Thus, we can define different data-flow abstractions of different resolutions (simple, distance vector, direction vector, and precise) in a similar manner as above.

Definition 2.2 *Let*

$\vec{s}, B, I, \vec{i}, \vec{i}', \vec{j}, \vec{j}', F$ and F' be same as that of definition 2.1, and $i'' \in I$, be an iteration,

A data-flow dependence abstraction, F , is defined by a partition function p_T on $I \times I$.

Iteration i' is flow dependent on i according to F iff $\exists \vec{j}, \vec{j}' \in I$ such that

$$F(\vec{j}, \vec{s}) = F'(\vec{j}', \vec{s}), \vec{j} \prec \vec{j}', p(\vec{i}, \vec{i}') = p(\vec{j}, \vec{j}'), \\ \nexists \vec{j}'' (\neq \vec{j}') \text{ such that } F'(\vec{j}'', \vec{s}) = F'(\vec{j}', \vec{s}), \vec{j} \prec \vec{j}'' \prec \vec{j}',$$

The partition functions P_{simple} , P_{level} , P_{dist} , P_{dir} and $P_{precise}$ applied to the flow-dependence approximation will generate *Simple Data-flow Dependence Analysis*, *Data-flow Levels*, *Data-flow Distance Vectors*, *Data-flow Direction Vectors* and *Precise Data-flow Dependence Pairs* respectively.

2.3 The family of approximations

We have defined the set of approximations such that they can be applied to both data dependence and data-flow problems. The resolution of each approximation is the number of equivalence classes in the partition generated by the equivalence function. Table 1 is a summary of the approximations. The resolution is given for m loop nests with each loop containing N iterations.

We can classify different researchers' analysis techniques using these sets of approximations. Much of the dependence analysis research has focused on distance and direction vectors. For a good set of references see [9]. More recent work has looked into simple data-flow analysis and data-flow vectors[3][15][14]. Other researchers have focused on extended traditional data-flow analysis to individual array elements[16][11][10].

Table 2 shows the results of analyzing two simple programs using all the different approximations discussed in this section.

<i>Number of equivalence classes</i>	<i>Data Dependence Problem</i>	<i>Data-Flow Dependence Problem</i>
1	Simple Data Dependence Analysis	Simple Data-flow Analysis
m	Dependence Levels	Data-flow Dependence Levels
3^m	Direction Vectors	Data-flow Direction Vectors
$(2N - 1)^m$	Distance Vectors	Data-flow Distance Vectors
N^{2m}	Precise Data Dependence Pairs	Precise Data-Flow Dependence Pairs

Table 1: Family of approximations

3 Precise Data Dependence Pairs

Ideally, we would like a representation that does not lose information. Storing a list of all dependent pairs is not possible at compile time due to the presence of symbolic constants. Even without symbolic terms, the number of dependent pairs could be much too large to be practical.

In this section, we describe a compact representation and analysis technique for the precise data dependence problem. We represent the precise data dependence information by a *reference tree*, RT. An RT is a relation between two array accesses F and F' . An RT maps an iteration, $\vec{i}$, of F into all the corresponding data dependent iterations, $\vec{i}'$, of F' that occur before $\vec{i}$.

Formally:

Definition 3.1

Let $\vec{i}, \vec{i}'$ be iterations, $\vec{s}$ be the symbolic constant vector,

$$\vec{u} = [i_1 \dots i_n, s_1 \dots s_p], \quad \vec{u}' = [i'_1 \dots i'_n, s_1 \dots s_p]$$

F and F' be affine access functions,

B and B' be affine bound constraints,

$\vec{l}$ be an array location,

$RT_{F, F', B, B'}(\vec{u}) = \text{all } (\vec{i}') \text{ such that}$

$$B\vec{u} \geq \vec{0} \quad B'\vec{u}' \geq \vec{0} \quad F\vec{u} = \vec{l} \quad F'\vec{u}' = \vec{l} \quad \vec{i}' \prec \vec{i}$$

We illustrate RTs with the following example:

```

for  $i_1 = 1$  to  $n$  do
  for  $i_2^1 = 1$  to  $n$  do
     $work[i_2^1] = \dots$ 
  for  $i_2^2 = 1$  to  $n$  do
     $\dots = work[i_2^2]$ 

```

We create an RT which maps all the values of $\vec{i}_r$ into the preceding dependent writes $\vec{i}_w$. Given an $\vec{i}_r$, in order for there to be a preceding dependent write, $i_{w2}^1 = i_{r2}^2$ and $i_{w1}^1 \leq i_{r1}^2$.

Examples	for $i = 0$ to M do for $j = 0$ to N do $A[j] = \dots$ $\dots = A[j]$	for $i = 0$ to L do for $j = 0$ to M do for $k = 0$ to N do $A[j] = A[j] + \dots$
<i>Type of analysis</i>	<i>Results of the analysis applied to the write access vs. the read access</i>	
Simple Data-Dependence	data dependent	data dependent
Dependence Levels	1, 3	1, 3, 4
Direction Vectors	$\begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} '+' \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ '+' \end{bmatrix} \begin{bmatrix} '+' \\ 0 \\ - \end{bmatrix} \begin{bmatrix} '+' \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} '+' \\ 0 \\ '+' \end{bmatrix}$
Distance Vectors	$\begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \dots \begin{bmatrix} M \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \dots \begin{bmatrix} 0 \\ 0 \\ N \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ -N \end{bmatrix} \dots \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \dots \begin{bmatrix} 1 \\ 0 \\ N \end{bmatrix} \dots$ $\dots \begin{bmatrix} L \\ 0 \\ -N \end{bmatrix} \dots \begin{bmatrix} L \\ 0 \\ -1 \end{bmatrix} \begin{bmatrix} L \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} L \\ 0 \\ 1 \end{bmatrix} \dots \begin{bmatrix} L \\ 0 \\ N \end{bmatrix}$
Precise Data Dependence Pairs	$\left\{ \left(\begin{bmatrix} i' \\ j \end{bmatrix}, \begin{bmatrix} i \\ j \end{bmatrix} \right) \mid \begin{array}{l} 0 \leq i \leq M \\ 0 \leq i' \leq i \\ 0 \leq j \leq N \end{array} \right\}$	$\left\{ \left(\begin{bmatrix} i \\ j \\ k' \end{bmatrix}, \begin{bmatrix} i \\ j \\ k \end{bmatrix} \right) \mid \begin{array}{l} 0 \leq i \leq L \\ 0 \leq j \leq M \\ 0 \leq k \leq N \\ 0 \leq k' \leq k \end{array} \right\}$ $\cup \left\{ \left(\begin{bmatrix} i' \\ j \\ k' \end{bmatrix}, \begin{bmatrix} i \\ j \\ k \end{bmatrix} \right) \mid \begin{array}{l} 0 \leq i \leq L \\ 0 \leq i' \leq i - 1 \\ 0 \leq j \leq M \\ 0 \leq k \leq N \\ 0 \leq k' \leq N \end{array} \right\}$
Simple Data-flow Dependence	data-flow dependent	data-flow dependent
Data-flow Dependence Levels	3	1, 3
Data-flow Direction Vectors	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ '+' \end{bmatrix} \begin{bmatrix} '+' \\ 0 \\ - \end{bmatrix}$
Data-flow Distance Vectors	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ -N \end{bmatrix}$
Precise Data-flow Dependence Pairs	$\left\{ \left(\begin{bmatrix} i \\ j \end{bmatrix}, \begin{bmatrix} i \\ j \end{bmatrix} \right) \mid \begin{array}{l} 0 \leq i \leq M \\ 0 \leq j \leq N \end{array} \right\}$	$\left\{ \left(\begin{bmatrix} i \\ j \\ k-1 \end{bmatrix}, \begin{bmatrix} i \\ j \\ k \end{bmatrix} \right) \mid \begin{array}{l} 0 \leq i \leq L \\ 0 \leq j \leq M \\ 1 \leq k \leq N \end{array} \right\}$ $\cup \left\{ \left(\begin{bmatrix} i-1 \\ j \\ N \end{bmatrix}, \begin{bmatrix} i \\ j \\ 0 \end{bmatrix} \right) \mid \begin{array}{l} 1 \leq i \leq L \\ 0 \leq j \leq M \end{array} \right\}$

Table 2: Results of the different analysis techniques for two simple examples

similar to LU decomposition, we decompose F' into $F'U = D$ where U is a unimodular integer matrix, and D is an integer echelon matrix. The initial system has an integer solution if and only if there exists an integer vector $\vec{t}$ such that $D\vec{t} = F'\vec{i}$. If such a $\vec{t}$ exists, then the solution is $\vec{i}' = U\vec{t}$.

To create the RT, we add in the additional $\vec{t}$ variables to our system. $D\vec{t} = F'\vec{i}$ is a set of integer constraints on the $\vec{t}$ and on the $\vec{i}$ variables. We add these constraints to our tree. If $D\vec{t} \neq F'\vec{i}$, there is no preceding dependent reference $\vec{i}'$, otherwise we create a tree as before, replacing $\vec{i}' = F'^{-1}F'\vec{i}$ with $\vec{i}' = U\vec{t}$.

In many cases, F' is invertible and F'^{-1} is an integer matrix. In such cases, we can use $\vec{i}' = F'^{-1}F'\vec{i}$ directly. There is no need for the auxiliary variables. Even in cases where F' is not invertible, it is sometimes possible to solve for some of the $\vec{t}$ variables. If for example we know that $t_1 = i'_1$, we can eliminate the use of t_1 by replacing all occurrences of t_1 with its value i'_1 .

4 Precise Data-Flow Dependence Pairs

When computing data-flow dependences, we are only interested in the last write before the read. We could define a *last reference tree* which would be analogous to our reference tree. As we are only interested in the last write before the read and not the last read before the write, we instead call the tree the *last write tree*, LWT.

Given a write statement and a read statement, the LWT is a function that maps read iterations into an expression for the last write iteration, before the read, to the same location. The last write before the read is the only write whose value is seen by the read.

Definition 4.1

Let $\vec{i}_w, \vec{i}'_w, \vec{i}_r$ be iterations, $\vec{s}$ be the symbolic constant vector,

$$\vec{u}_w = [i_{w1} \dots i_{wn}, s_1 \dots s_p], \quad \vec{u}'_w = [i'_{w1} \dots i'_{wn}, s_1 \dots s_p], \quad \vec{u}_r = [i_{r1} \dots i_{rn}, s_1 \dots s_p]$$

F_w and F_r be affine read and write access functions,

B_w and B_r be affine read and write loop bound constraints,

$\vec{l}$ be an array location,

$LWT_{F_w, F_r, B_w, B_r}(\vec{u}_r) = \vec{i}_w$ if and only if

$$\begin{aligned} B_w \vec{u}_w \geq \vec{0} \quad B_w \vec{u}_r \geq \vec{0} \quad F_r \vec{u}_w = \vec{l} \quad F_r \vec{u}_r = \vec{l} \quad \vec{i}_w \prec \vec{i}_r \\ \nexists \vec{i}'_w (\neq \vec{i}_w) \quad \text{such that} \quad B_w \vec{u}'_w \geq \vec{0} \quad F_r \vec{u}'_w = \vec{l} \quad \vec{i}_w \prec \vec{i}'_w \prec \vec{i}_r \end{aligned}$$

For example:

```

for  $i_1 = 1$  to  $n$  do
  for  $i_2 = 1$  to  $n$  do
    for  $i_3^1 = 1$  to  $i_2 + 10$  do
       $work[i_3^1] = \dots$ 
    for  $i_3^2 = 1$  to  $i_2 + 10$  do
       $\dots = work[i_3^2 + 1]$ 

```


all solutions with a level k dependence represent read iterations sets which do not have higher level dependences. This procedure generates the full LWT.

We have implicitly assumed that the read statement comes lexically before the write statement in the program text. If the write statement comes before the read, it is possible that there is a true dependence when $\vec{i}_w = \vec{i}_r$. We can consider this to be a level $n + 1$ dependence. In this case, the algorithm is slightly modified to start the recursion at level $n + 1$ instead of at level n .

We have implemented these algorithms and find them complex and inefficient. Each Parametric Integer Programming problem requires solving Integer Programming problems, the basis of affine memory disambiguation, many times. For each array reference pair, we are required to solve exponentially many Parametric Integer Programming problems for each dependence level.

4.2 LWTs for Multiple Reads and Writes

We have described an algorithm to calculate the LWT for a single pair of read and write statements. For multiple read statements we can construct an LWT for each read statement and treat them as separate problems. When there are multiple writes we need to construct a single LWT that will give the last write before the read of all the writes for every location. In general this requires us to intersect the LWTs, comparing each leaf of one with each leaf of the other, a process that is exponential in the number of write statements. In practice this can be done more efficiently for special cases such as when the array writes do not write to overlapping locations.

4.3 Efficiently Calculating Last Write Tree

We believe that Feautrier’s algorithm is too inefficient for general use. We have previously shown that reference trees can be constructed very efficiently. In this section, we define a write tree, WT, to be any RT that maps an iteration, $\vec{i}$ of a *read* access, into all earlier data dependent iterations, $\vec{i}'$, of a *write* access. We shall show that in the common cases seen in practice, WTs provide us with an efficient method to compute LWTs.

We say that a write statement *self interferes* if it writes to the same memory location on multiple iterations. If a write never self interferes, there is at most one write for every location and the WT is an LWT. This condition is very restrictive. Frequently, though, in real programs, some of the loop indices are not used by a self interfering write. We define a loop index to be unused if it does not appear in the array reference function, F_w , nor in the bounds of any used variable. We can temporarily eliminate these variables, creating a reduced system. If in this reduced system, the write statement does not self interfere, the LWT for the reduced system is equivalent to the WT for the reduced system. We will show that given this LWT, we can efficiently construct the LWT for the original system. Thus, if there is no interference after removing the unused variables, we can efficiently construct the LWT.

We tested this hypothesis by looking at all of the affine array writes in 12 of the PERFECT Club programs which were nested in at least one loop. After eliminating the unused variables, we checked to see if each write statement self-interfered, if it wrote the same location multiple times. This is equivalent to checking for an output dependence from the write to itself with a non "0" direction vector component. In Table 3 we show the results. After removing the unused variables, in 2,412 out of 2,476 writes, over 97%, the writes do not self interfere.

Self Interference													
	APS	LGS	LWS	MTS	NAS	OCS	SDS	SMS	SRS	TFS	TIS	WSS	TOTAL
Self Interference	0	0	0	0	13	39	0	0	0	0	12	0	64
No Self Interference	424	36	36	74	316	75	142	124	425	233	10	517	2,412

Table 3: Number of writes that self interfere after removing unused variables

Given an LWT for the reduced system, we can efficiently construct an LWT for the original system. As an example, assume we have a write statement nested in four loops; i_{w_1} , i_{w_2} , i_{w_3} and i_{w_4} . Assume that variables i_{w_2} and i_{w_3} are unused. We temporarily remove the unused variables. Assume that the write in this reduced system does not self interfere. We create a WT (which is equivalent to the LWT) for this reduced system as if variables i_{w_2} and i_{w_3} did not exist. We show this tree in Figure 4. We label some of the internal nodes $N_1 \dots N_4$ to aid in the discussion.

Since i_{w_2} and i_{w_3} are unused, the write operation will write to the same location regardless of their value. Therefore, in converting this tree into our full LWT, we are simply interested in setting i_{w_2} and i_{w_3} to the latest possible value that is still before the read. We can accomplish this goal with the LWT in Figure 5.

Traversing the reduced tree in a depth first manner, we made the following modifications to convert it into our full LWT.

For the level 1 dependence, we know that $i_{w_1} < i_{r_1}$. Whatever values we give the unused write variables, the write will come before the read. Therefore, we set the unused variables to be as large as possible, their upper bounds. For triangular loops, we find the maximum upper bounds over the triangle using a simple substitution algorithm. This substitution algorithm requires each instance of a loop to always contain at least one iteration. We can guarantee this condition by eliminating degenerate iterations using an algorithm based on Fourier-Motzkin elimination[1]. In fact, such triangular degeneracies are very rare, and we found in 98% of the PERFECT Club writes that a simple substitution algorithm can be used to prove that no degeneracies exist.

In the reduced tree, the left child of node N_3 was $\perp$. This corresponds to the case when $i_{w_1} = i_{r_1}$ and $i_{w_4} > i_{r_4}$. In the reduced system, there is no such write before the read, but in the full system as long as $(i_{w_2}, i_{w_3}) \prec (i_{r_2}, i_{r_3})$, the write does come before the read.

The last write in this case is usually the level 3 dependence such that $i_{w_2} = i_{r_2}$ and $i_{w_3} = i_{r_3} - 1$, but it is not always legal to set i_{w_3} to $i_{r_3} - 1$. Since we only privatize common loops, the bounds on $\vec{i}_w$ up to the level of privatization are the same as the bounds on $\vec{i}_r$. If i_{r_3} is its minimum value, setting $i_{w_3} = i_{r_3} - 1$ would set the write iteration to a value less than its lower bound. If this is the case, we must try the level 2 dependence, $i_{w_2} = i_{r_2} - 1$ and $i_{w_3} = UB_3(i_{w_1}, i_{w_2})$, where $UB_3(i_{w_1}, i_{w_2})$ is an expression for the upper bound of i_{w_3} . If i_{r_2} is at its minimum value, this assignment is also illegal and there is no write before the read for this read iteration set.

Continuing our traversal, we come to the level 4 dependence. For this dependence, we know that $i_{w_1} = i_{r_1}$ and that $i_{w_4} < i_{r_4}$. The last iteration before the read occurs when $i_{w_2} = i_{r_2}$ and $i_{w_3} = i_{r_3}$.

Finally, we come to the right child of node N_4 . This corresponds to the case when $i_{w_1} = i_{r_1}$ and $i_{w_4} = i_{r_4}$. This case is exactly equivalent to the left child of node N_3 , and we insert the same subtree here.

The properties of our LWTs can be exploited in optimizing this procedure. Burke and Cytron’s method works by enumerating and verifying possible direction vectors. If the dependence level of a leaf is k , any data-flow vector with a direction other than 0 in any of its first $k - 1$ dimensions is infeasible. We can immediately prune away any such vector. Given an LWT, we can calculate data-flow vectors this way as efficiently as calculating direction vectors.

6 Array Privatization: An Optimization

We will demonstrate the necessity of more powerful data dependence representations with the array privatization example. Recent research by Eigenmann et al.[5] and by us[13] has shown that array privatization is a critical transformation in the suite of parallelizing transformations[13][5]. Eigenmann et al. discuss the techniques used to hand-parallelize four of the PERFECT Club programs[6]. They find that array privatization is useful in all of the programs. In two of the four programs, 98% of the dynamic execution time of the program is spent in loops that require array privatization. In order to privatize arrays, traditional data dependence techniques are not sufficient; data-flow dependence vectors are required.

This section illustrates a compiler algorithm for array privatization in the domain of simple loop nests that contain no IF statements and no procedure calls. A more detailed and formal description is found in[12].

Consider the example in Figure 6 extracted from the PERFECT Club benchmark program OCS. We will use this example throughout this section to illustrate the array privatization problem. Every iteration of the outer loop reads and writes the same elements of the array *work*. It can be shown in general that any loop, k , can be parallelized using privatization if the loop has no k -level data-flow vectors. In this example, the data-flow distance vector between any two references to the *work* array is (0). There is no level 1 data-flow dependence, and therefore the loop can be parallelized with privatization. Using just direction vectors, a system would detect a dependence across the iterations of the outermost loop. The outermost loop would therefore not be parallelizable.

```

for  $j = 1$  to  $N1$  by  $2$  do
  for  $i = 1$  to  $N2P$  do
     $ii = i + i$ 
     $work[ii - 1] = A[i][j]$ 
     $work[ii] = A[i][j + 1]$ 
  for  $i = 1$  to  $N2P$  do
     $A[i][j] = work[i]$ 
     $A[i][j + 1] = work[i + N2P]$ 

```

Figure 6: Example of a Privatizable Array

Privatizability implies that no communication is necessary between iterations of the loop, but each processor may need to initialize its private copy and to copy back some data from the private copy to the original variable at the end of the execution. We refer to the former process *initialization* and the latter *finalization*.

In our example, every work location read is initialized in the loop. We could modify the example slightly so that some of the data being read is initialized outside of the loop.

```

work[0] = 3
for j = 1 to N1 by 2 do
  for i = 1 to N2P do
    ii = i + i
    work[ii - 1] = A[i][j]
    work[ii] = A[i][j + 1]
  for i = 0 to N2P do
    A[i][j] = work[i]
    A[i][j + 1] = work[i + N2P]

```

In the modified example, location work[0] is written before the start of the loop. It is still legal to privatize; the only data-flow dependence vector is still (0). Now, though, we must initialize each processor's local value of work[0] with the global value of work[0].

In general we need the full power of LWTs to calculate which elements need to be initialized. Any array element read for which the last write before the read is not in the same iteration, must be initialized. For initialization, we can always be conservative and initialize the entire array.

Note, that for this example, it is very difficult to discover that initialization is unnecessary. Each read gets part of its data from each write. Therefore, we can not simply look at pairs of references. The effects of the two writes must be combined by intersecting the two individual LWTs.

For both the original example of figure 6 and the modified version, we must copy some of the values from the local arrays back to the global after executing the loop. For these examples, the only values seen after the loop are written in the last iteration, $j = N1$. Thus, we can strip the last iteration and have the processor assigned to this iteration write to the global array rather than to its local array. It can be shown that for the cases where our efficient algorithm for calculating LWTs applies, it is always possible to finalize simply by stripping the last iteration.

We have implemented our algorithm in the SUIF compiler system. Our parallelizer marks all the parallelizable loops and privatizes arrays when needed. It privatizes using our efficient algorithm for calculating LWTs. To show the results of the implementation of our algorithm, we show the output of our parallelizer on the example.

The SUIF optimizer did the normalization, constant propagation and induction variable identification. Our parallelizer generated the following code which is run on each processor. To simplify the presentation, we do not parallelize the inner loops although they can be parallelized using standard techniques.

```

LowerBound = 0
UpperBound = divfloor(N1-1,2)

```



```

BlockSz = divceil(UpperBound-LowerBound,NumProc)
/* Initialization */
for i = 1 to 100 do
    _work_priv[i] = work[i]
endfor
AtomicIncrement(InitCount) /* Synchronization with finalization */
/* Body */
if (ProcID!=NumProc-1) then /* not the last processor */
    for j = LowerBound+ProcID*BlockSz to LowerBound+(ProcID+1)*BlockSz - 1 do
        for i = 1 to N2P do
            _work_priv[2*i-1] = A[i][2*j+1]
            _work_priv[2*i] = A[i][2*j+2]
        end for
        for i = 1 to N2P do
            A[i][2*j+1] = _work_priv[i]
            A[i][2*j+2] = _work_priv[i+N2P]
        end for
    end for
else /* last processor */
    for j = LowerBound+ProcID*BlockSz to UpperBound - 1 do
        for i = 1 to N2P do
            _work_priv[2*i-1] = A[i][2*j+1]
            _work_priv[2*i] = A[i][2*j+2]
        end for
        for i = 1 to N2P do
            A[i][2*j+1] = _work_priv[i]
            A[i][2*j+2] = _work_priv[i+N2P]
        end for
    end for
    /* Wait for all initialization to finish */
    while InitCount < NumProc do busywait endwhile
    /* Finalization */
    for i = 1 to N2P do
        work[2*i-1] = A[i][2*UpperBound+1]
        work[2*i] = A[i][2*UpperBound+2]
    end for
    for i = 1 to N2P do
        A[i][2*UpperBound+1] = work[i]

```

```

    A[i][2*UpperBound+2] = work[i+N2P]
end for

```

Note that our system conservatively initializes the entire array. We do not currently intersect the LWTs of two write statements. Also note that the only synchronization needed is that the finalization should execute after all the initializations.

7 Conclusions

Standard data dependence abstractions techniques are not sufficient for the more advanced optimizations being considered today. In particular, scalar optimizers have long used both data-flow and alias information to analyze programs. Parallelizers traditionally use only data dependence analysis or, more precisely, affine memory disambiguation of array locations. A read statement is assumed to be dependent on a write if the read and write statements access overlapping locations. Data-flow analysis on individual array elements allows for more powerful optimizations.

In this paper, we propose a unified framework to describe different data dependence abstractions. We show how standard representations such as distance and direction vectors fit into this framework. In addition, we propose new representations such as data-flow dependence vectors, RTs and LWTs.

As an example, this paper demonstrates the successful application of array data-flow analysis in array privatization. Using data-flow vectors and LWTs, we can easily integrate array privatization into parallelization. Replacing data-dependence vectors with data-flow vectors, the spurious dependences due to the reuse of the array locations are eliminated.

We show how to derive data-flow direction vectors from a last write tree (LWT), a general representation capturing the complete results from array data-flow analysis. Most array references are simple. Our empirical results suggest that most arrays either exhibit no reuse, or the array reuse occurs within those loops whose indices are unused by the array write reference function. This observation leads to an algorithm that can efficiently generate a last write tree and data-flow dependence vectors for these common cases.

This paper lays the foundation for new compiler optimizations that use sophisticated data-flow analysis on array elements. We have studied array privatization in this paper, focusing on the new analysis techniques. In addition to array privatization, our analysis techniques can also be used for other loop transformations based on dependence vectors, such as unimodular loop transformations and tiling or blocking. Data-flow analysis on arrays will become even more important when compiling for distributed memory machines.

References

- [1] C. Ancourt and F. Irigoin. Scanning polyhedra with do loops. In *Proceedings of the ACM SIGPLAN 1991 PPOPP*, 1991.

- [2] U. Banerjee. *Dependence Analysis for Supercomputing*. Kluwer Academic, 1988.
- [3] T. Brandes. The importance of direct dependences for automatic parallelism. In *Proceedings of 1988 International Conference on Supercomputing*, 1988.
- [4] M. Burke and R. Cytron. Interprocedural dependence analysis and parallelization. In *Proceedings of the SIGPLAN 1986 Symposium on Compiler Construction*, pages 162–175, 1986.
- [5] R. Eigenmann, J. Hoeflinger, Z. Li, and D. Padua. Experience in the automatic parallelization of four perfect-benchmark programs. In *Proceedings of the Fourth Workshop on Languages and Compilers for Parallel Computing*, 1991.
- [6] M. Berry et al. The PERFECT Club benchmarks: effective performance evaluation of supercomputers. Technical Report UIUCSRD Rep. No. 827, University of Illinois Urbana-Champaign, 1989.
- [7] P. Feautrier. Array expansion. In *International Conference on Supercomputing*, pages 429–442, 1988.
- [8] P. Feautrier. Parametric integer programming. Technical Report 209, Laboratoire Methodologie and Architecture Des Systemes Informatiques, Jan 1988.
- [9] G. Goff, K. Kennedy, and C. Tseng. Practical dependence testing. In *Proceedings of the SIGPLAN 1991 PLDI*, pages 15–29, 1991.
- [10] E.D. Granston and A.V. Veidenbaum. Detecting redundant accesses to array data. In *Supercomputing '91*, pages 854–865, 1991.
- [11] T. Gross and P. Steenkiste. Structured dataflow analysis for arrays and its use in an optimizing compiler. *Software - Practice and Experience*, 20(2):133–155, 1990.
- [12] D.E. Maydan, S. P. Amarasinghe, and M.S. Lam. Array data-flow analysis and its application in array privatization. In *submitted*, 1992.
- [13] D.E. Maydan, J.L. Hennessy, and M.S. Lam. Effectiveness of data dependence analysis. In *Proceedings of the NSF-NCRD Workshop on Advanced Compilation Techniques for Novel Architectures*, 1992.
- [14] W. Pugh and D. Wonnacott. Eliminating false data dependences using the omega test. In *Proceedings of the SIGPLAN 1992 PLDI*, pages 140–151, 1992.
- [15] H. Ribas. Obtaining dependence vectors for nested-loop computations. In *Proceedings of 1990 International Conference on Parallel Processing*, 1990.
- [16] C. Rosend. *Incremental Dependence Analysis*. PhD thesis, Rice University, March 1990.